

Chapter 5

Sorting Algorithms

Internal sort:

If the data sorting process takes place **entirely within the Random-Access Memory (RAM) of a computer**, it's called internal sorting. This is possible whenever the size of the dataset to be sorted is small enough to be held in RAM.

- Example: heap sort, bubble sort, selection sort, quick sort, shell sort, insertion sort.

External sort:

For sorting larger datasets, it may be necessary to hold only a smaller chunk of data in memory at a time, since it won't all fit in the RAM. The rest of the data is normally held on some larger, but slower medium, like a hard disk. **The sorting of these large datasets will require different sets of algorithms which are called external sorting.**

- Example: Merge sort, Tag sort, Polyphase sort, Four tape sort, External radix sort, Internal merge sort, etc.

What is stable sorting?

- When two same data appear in the **same order** in sorted data without changing their position is called stable sort.
- Example: *merge sort, insertion sort, bubble sort.*

What is Unstable sorting?

- When two same data appear in the **different order** in sorted data it is called unstable sort.
- Example: *quick sort, heap sort, shell sort.*

Insertion Sort Algorithm

Insertion sort is a sorting algorithm that places an unsorted element at its suitable place in each iteration.

Insertion sort works similarly as we sort cards in our hand in a card game.

We assume that the first card is already sorted then, we select an unsorted card. If the unsorted card is greater than the card in hand, it is placed on the right otherwise, to the left. In the same way, other unsorted cards are taken and put in their right place.

A similar approach is used by insertion sort.

Working of Insertion Sort

Suppose we need to sort the following array.



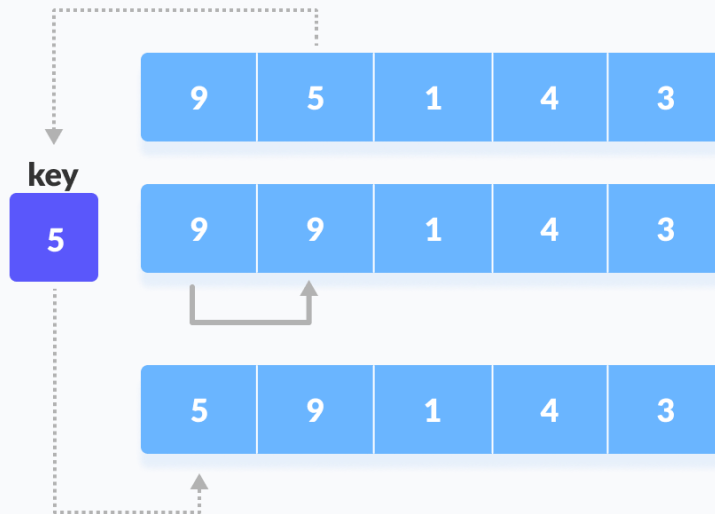
Initial array

1. The first element in the array is assumed to be sorted. Take the second element and store it separately in `key`.

Compare `key` with the first element. If the first element is greater than `key`,

then **key** is placed in front of the first element.

step = 1

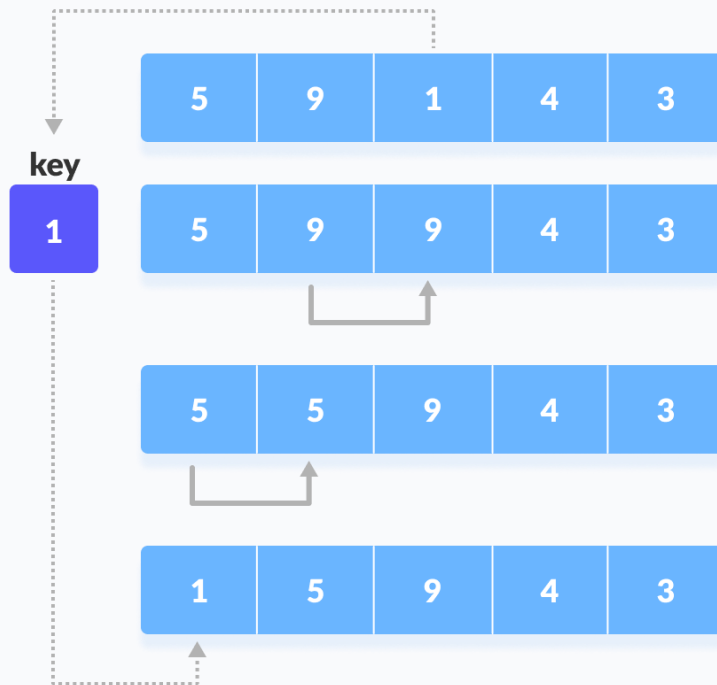


2. If the first element is greater than key, then key is placed in front of the first element.
3. Now, the first two elements are sorted.

Take the third element and compare it with the elements on the left of it.
Placed it just behind the element smaller than it. If there is no element

smaller than it, then place it at the beginning of the array.

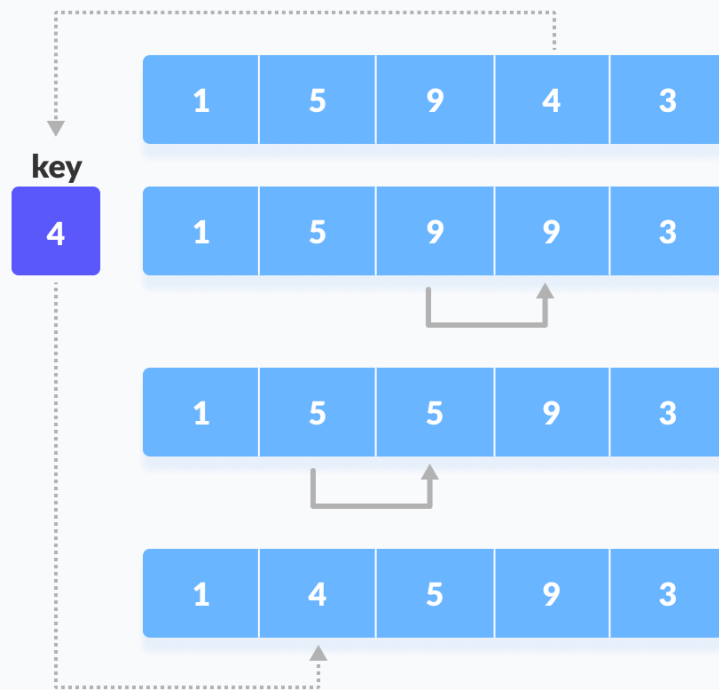
step = 2



4. Place 1 at the beginning

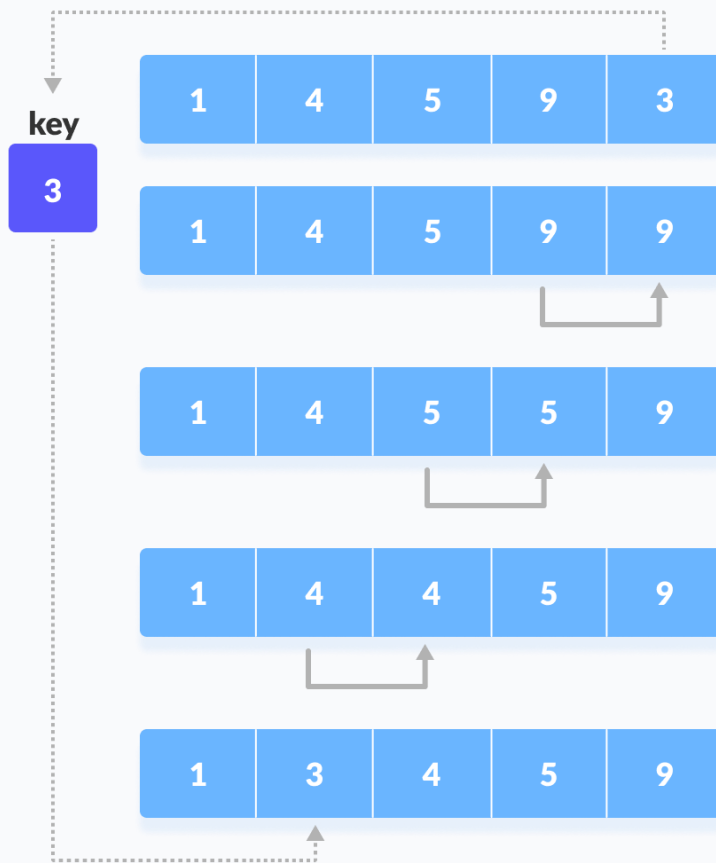
5. Similarly, place every unsorted element at its correct position.

step = 3



6. Place 4 behind 1

step = 4



7. Place 3 behind 1 and the array is sorted

Illustration

$arr = \{23, 1, 10, 5, 2\}$

Initial:

- Current element is **23**
- The first element in the array is assumed to be sorted.
- The sorted part until **0th** index is : **[23]**

First Pass:

- Compare **1** with **23** (current element with the sorted part).

- Since **1** is smaller, insert **1** before **23** .
- The sorted part until **1st** index is: **[1, 23]**

Second Pass:

- Compare **10** with **1** and **23** (current element with the sorted part).
- Since **10** is greater than **1** and smaller than **23** , insert **10** between **1** and **23** .
- The sorted part until **2nd** index is: **[1, 10, 23]**

Third Pass:

- Compare **5** with **1** , **10** , and **23** (current element with the sorted part).
- Since **5** is greater than **1** and smaller than **10** , insert **5** between **1** and **10**
- The sorted part until **3rd** index is : **[1, 5, 10, 23]**

Fourth Pass:

- Compare **2** with **1, 5, 10** , and **23** (current element with the sorted part).
- Since **2** is greater than **1** and smaller than **5** insert **2** between **1** and **5** .
- The sorted part until **4th** index is: **[1, 2, 5, 10, 23]**

Final Array:

- The sorted array is: **[1, 2, 5, 10, 23]**

Complexity Analysis of Insertion Sort :

Time Complexity of Insertion Sort

- **Best case: $O(n)$** , If the list is already sorted, where n is the number of elements in the list.
- **Average case: $O(n^2)$** , If the list is randomly ordered
- **Worst case: $O(n^2)$** , If the list is in reverse order

Space Complexity of Insertion Sort

- **Auxiliary Space: $O(1)$** , Insertion sort requires **$O(1)$** additional space, making it a space-efficient sorting algorithm.

Advantages of Insertion Sort:

- Simple and easy to implement.
- **Stable** sorting algorithm.
- Efficient for small lists and nearly sorted lists.
- Space-efficient as it is an in-place algorithm.

- Adoptive. the number of inversions is directly proportional to number of swaps. For example, no swapping happens for a sorted array and it takes $O(n)$ time only.

Disadvantages of Insertion Sort:

- Inefficient for large lists.
- Not as efficient as other sorting algorithms (e.g., merge sort, quick sort) for most cases.

Insertion Sort Applications

The insertion sort is used when:

- the array is has a small number of elements
- there are only a few elements left to be sorted

Selection Sort

-

Selection Sort is a comparison-based sorting algorithm. It sorts an array by repeatedly selecting the **smallest (or largest)** element from the unsorted portion and swapping it with the first unsorted element. This process continues until the entire array is sorted.

1. First we find the smallest element and swap it with the first element. This way we get the smallest element at its correct position.
2. Then we find the smallest among remaining elements (or second smallest) and swap it with the second element.
3. We keep doing this until we get all elements moved to correct position.

Working of Selection Sort

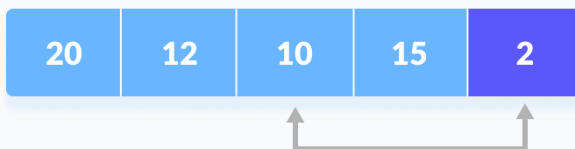
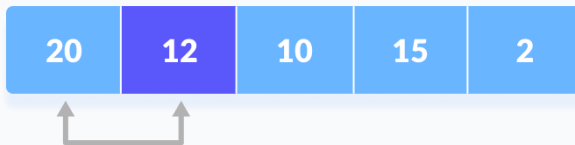


1. Set the first element as `minimum`.

Select first element as minimum

2. Compare `minimum` with the second element. If the second element is smaller than `minimum`, assign the second element as `minimum`.

Compare `minimum` with the third element. Again, if the third element is smaller, then assign `minimum` to the third element otherwise do nothing. The process goes on until the last element.



Compare minimum with the

remaining elements

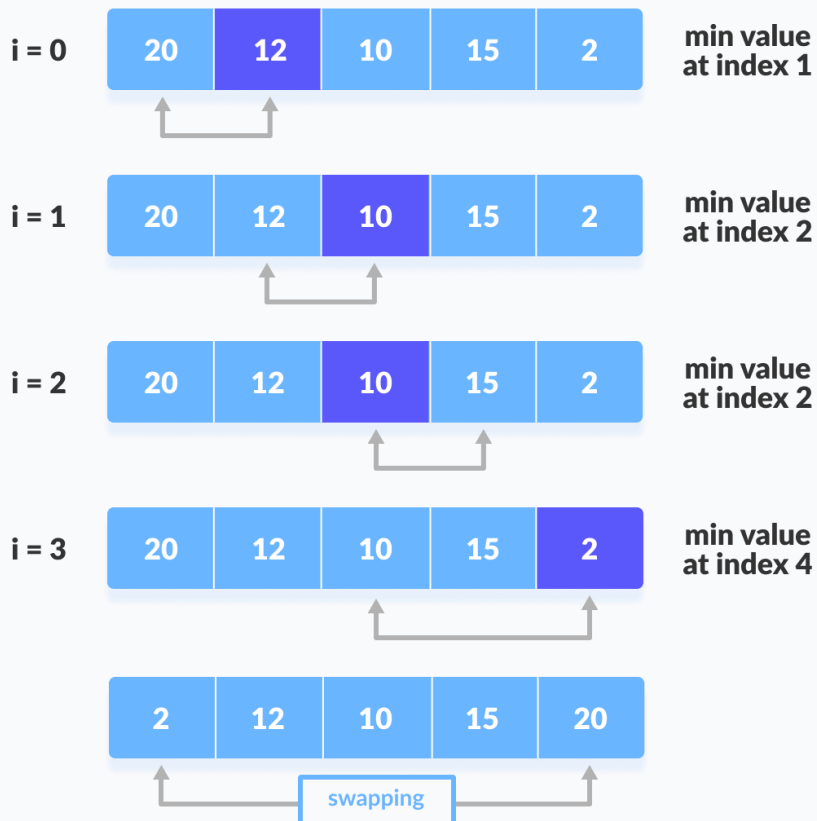
3. After each iteration, `minimum` is placed in the front of the unsorted list.



4. Swap the first with minimum

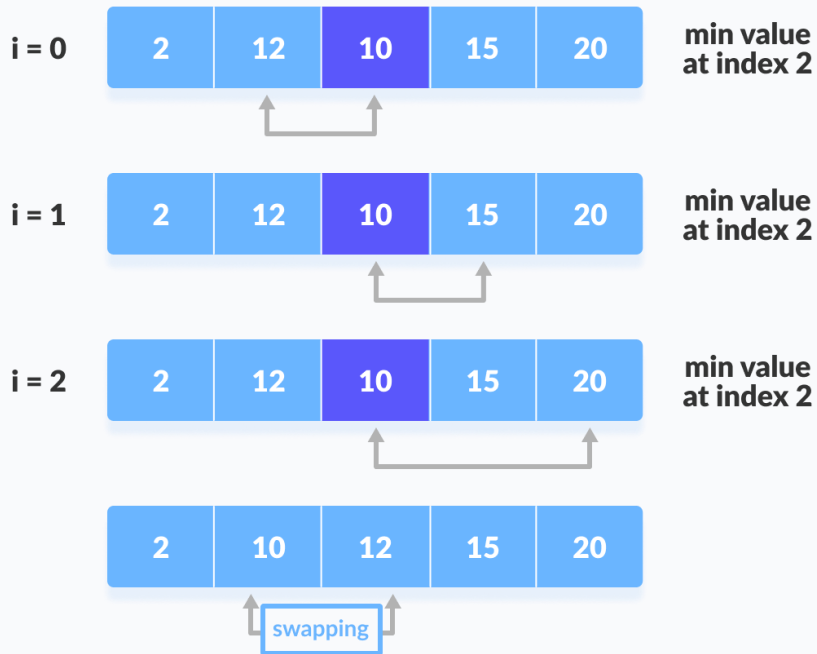
5. For each iteration, indexing starts from the first unsorted element. Step 1 to 3 are repeated until all the elements are placed at their correct positions.

step = 0



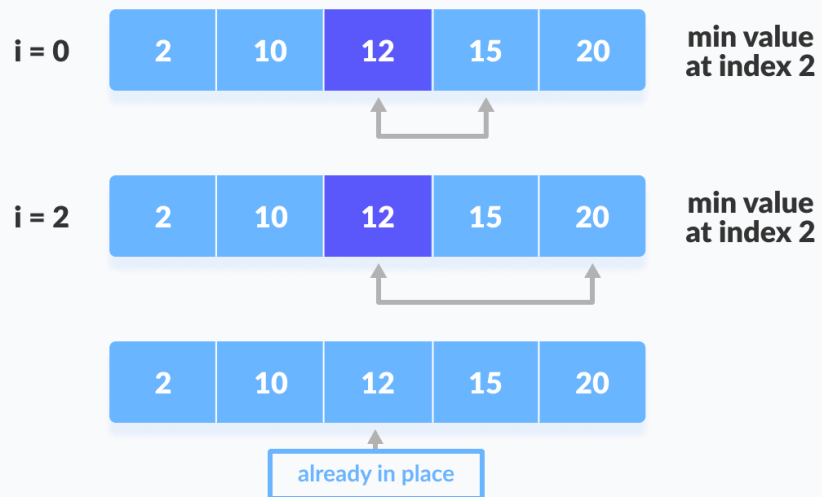
6. The first iteration

step = 1



7. The second iteration

step = 2



The third iteration

step = 3



8. The fourth iteration

Complexity Analysis of Selection Sort

Time Complexity: $O(n^2)$, as there are two nested loops:

- One loop to select an element of Array one by one = $O(n)$
- Another loop to compare that element with every other Array element = $O(n)$
- Therefore overall complexity = $O(n) * O(n) = O(n*n) = O(n^2)$

Auxiliary Space: $O(1)$ as the only extra memory used is for temporary variables.

Advantages of Selection Sort

- Easy to understand and implement, making it ideal for teaching basic sorting concepts.
- Requires only a constant $O(1)$ extra memory space.
- It requires less number of swaps (or memory writes) compared to many other standard algorithms. Only cycle sort beats it in terms of memory writes. Therefore it can be simple algorithm choice when memory writes are costly.

Disadvantages of the Selection Sort

- Selection sort has a time complexity of $O(n^2)$ makes it slower compared to algorithms like Quick Sort or Merge Sort.
- Does not maintain the relative order of equal elements which means it is not stable.

Bubble Sort Algorithm

- **Bubble Sort** is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets as its average and worst-case time complexity are quite high.

- We sort the array using multiple passes. After the first pass, the maximum element goes to end (its correct position). Same way, after second pass, the second largest element goes to second last position and so on.
- In every pass, we process only those elements that have already not moved to correct position. After k passes, the largest k elements must have been moved to the last k positions.
- In a pass, we consider remaining elements and compare all adjacent and swap if larger element is before a smaller element. If we keep doing this, we get the largest (among the remaining elements) at its correct position.

Working of Bubble Sort

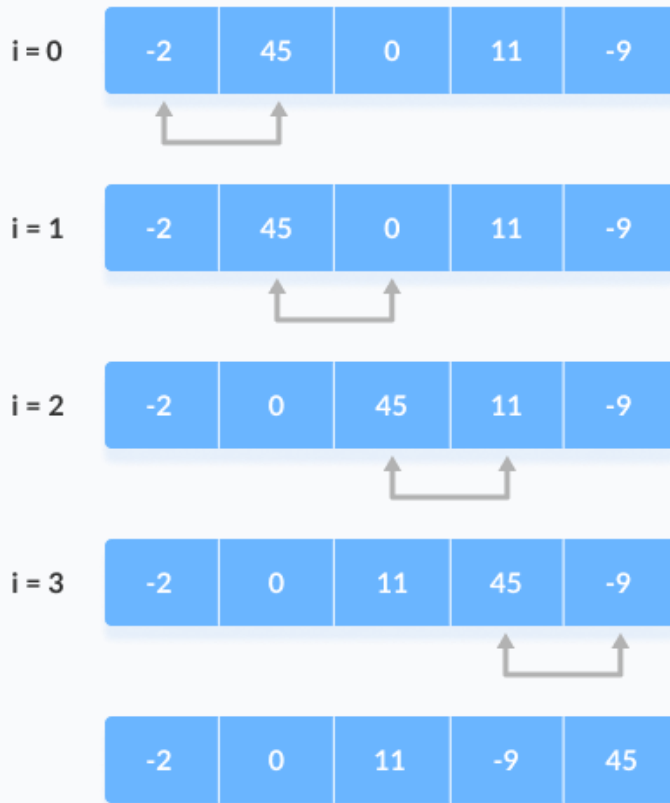
Suppose we are trying to sort the elements in **ascending order**.

1. First Iteration (Compare and Swap)

1. Starting from the first index, compare the first and the second elements.
2. If the first element is greater than the second element, they are swapped.
3. Now, compare the second and the third elements. Swap them if they are not in order.

4. The above process goes on until the last element.

step = 0



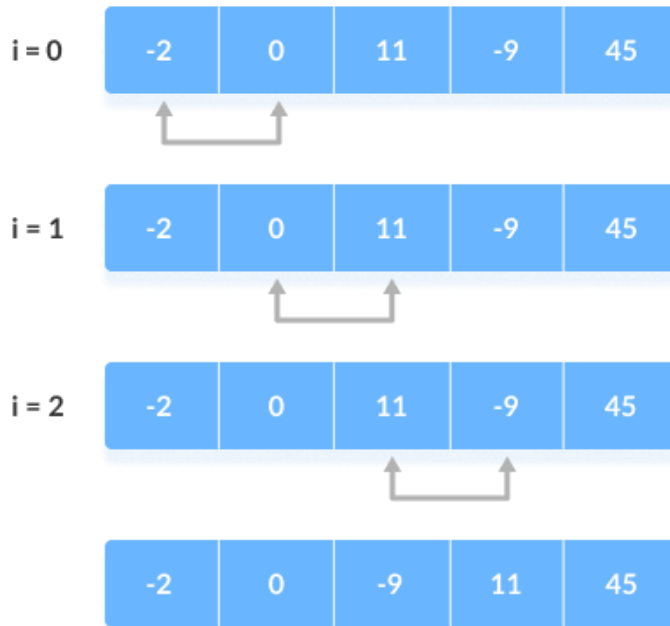
5. Compare the Adjacent Elements

2. Remaining Iteration

The same process goes on for the remaining iterations.

After each iteration, the largest element among the unsorted elements is placed at the end.

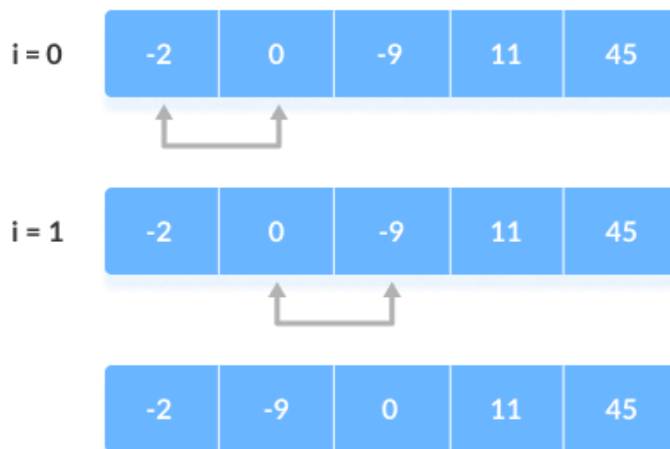
step = 1



Put the largest element at the end

In each iteration, the comparison takes place up to the last unsorted element.

step = 2

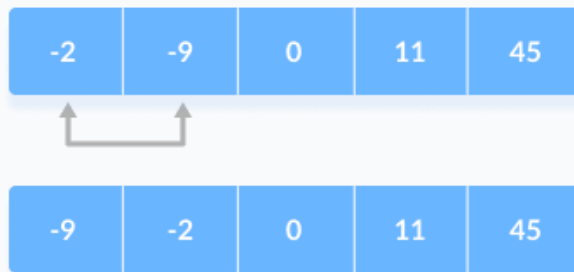


Compare the adjacent elements

The array is sorted when all the unsorted elements are placed at their correct positions.

step = 3

i = 0



The array is sorted if all elements are kept in the right order

Complexity Analysis of Bubble Sort:

Time Complexity: $O(n^2)$

Auxiliary Space: $O(1)$

Advantages of Bubble Sort:

- Bubble sort is easy to understand and implement.
- It does not require any additional memory space.
- It is a stable sorting algorithm, meaning that elements with the same key value maintain their relative order in the sorted output.

Disadvantages of Bubble Sort:

- Bubble sort has a time complexity of $O(n^2)$ which makes it very slow for large data sets.
- Bubble sort is a comparison-based sorting algorithm, which means that it requires a comparison operator to determine the relative order of elements in the input data set. It can limit the efficiency of the algorithm in certain cases.

Exchange Sort

- The exchange sort compares each element of an array and swaps those elements that are not in their position.
- The exchange sort compares the first element with each element of array, and makes swap whenever it is necessary.
- This technique is similar to bubble sort but there is slightly difference in the matter they compared the element.

Example:

Step 1: Select index 0 element, and compare it with all other elements in the array with higher index, and swap when it is not in order.

Let us consider the following array:

4	8	9	6	2	5
---	---	---	---	---	---

Iteration 1: Compare 4 with 8, no exchange.

4	8	9	6	2	5
---	---	---	---	---	---

Iteration 2: Compare 4 with 9, no exchange.

4	8	9	6	2	5
---	---	---	---	---	---

Iteration 3: Compare 4 with 6 , no exchange

4	8	9	6	2	5
---	---	---	---	---	---

Iteration 4: Compare 4 with 2, exchange

4	8	9	6	2	5
---	---	---	---	---	---

After exchange

2	8	9	6	4	5
---	---	---	---	---	---

Iteration 5: Compare 2 with 5, no exchange.

2	8	9	6	4	5
---	---	---	---	---	---

In the first round 2 is sorted.

Step 2: Select index 1 element (8) and compare it with all elements to the right.

Iteration 1: Compare 8 with 9, no exchange.

Array:

2 8 9 6 4 5

Iteration 2: Compare 8 with 6, exchange.

Array:

2 6 9 8 4 5

Iteration 3: Compare 6 with 4, exchange.

Array:

2 4 9 8 6 5

Iteration 4: Compare 4 with 5, no exchange.

Array remains the same:

2 4 9 8 6 5

At the end of this round, **4** is sorted in its correct position.

Step 3: Select index 2 element (9) and compare it with all elements to the right.

Iteration 1: Compare 9 with 8, exchange.

Array:

2 4 8 9 6 5

Iteration 2: Compare 8 with 6, exchange.

Array:

2 4 6 9 8 5

Iteration 3: Compare 6 with 5, exchange.

Array:

2 4 5 9 8 6

At the end of this round, **5** is sorted in its correct position.

Step 4: Select index 3 element (9) and compare it with all elements to the right.

Iteration 1: Compare 9 with 8, exchange.

Array:

2 4 5 8 9 6

Iteration 2: Compare 8 with 6, exchange.

Array:

2 4 5 6 9 8

At the end of this round, **6** is sorted in its correct position.

Step 5: Select index 4 element (9) and compare it with the last element.

Iteration 1: Compare 9 with 8, exchange.

Array:

2 4 5 6 8 9

At the end of this round, **8** is sorted in its correct position.

Step 6: Final Step

At this point, all elements are sorted. The sorted array is:

2 4 5 6 8 9

Final Observations:

- In each step, the smallest element (from the unsorted portion of the array) is moved to its correct position.
- The process is repeated until the entire array is sorted.

Complexity:

- **Time Complexity:** $O(n^2)$, because for each element, the algorithm compares it with all other elements in the unsorted portion of the array.
- **Space Complexity:** $O(1)$, as it sorts the array in place without requiring extra space.

Quick Sort

QuickSort is a sorting algorithm based on the Divide and Conquer that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array.

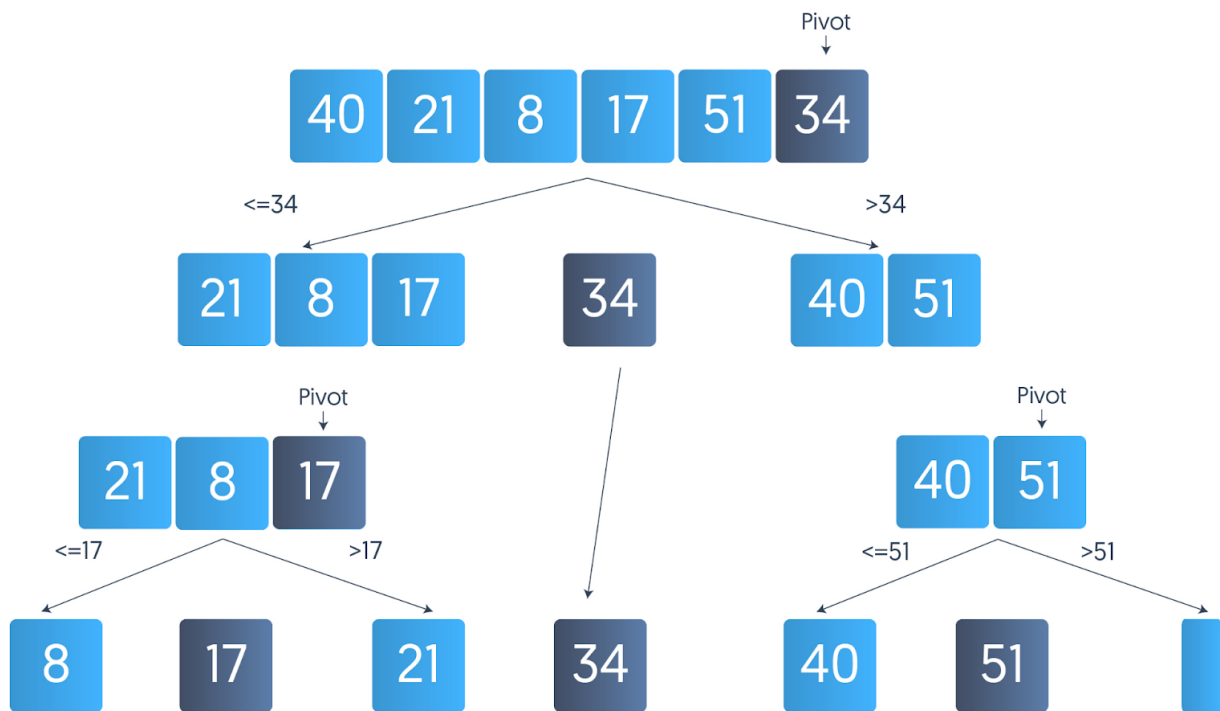
Quicksort is a sorting algorithm based on the **divide and conquer approach** where

1. An array is divided into subarrays by selecting a **pivot element** (element selected from the array).

While dividing the array, the pivot element should be positioned in such a way that elements less than pivot are kept on the left side and elements greater than pivot are on the right side of the pivot.

2. The left and right subarrays are also divided using the same approach. This process continues until each subarray contains a single element.
3. At this point, elements are already sorted. Finally, elements are combined to form a sorted array.

Working of Quicksort Algorithm



1. Select the Pivot Element

There are different variations of quicksort where the pivot element is selected from different positions. Here, we will be selecting the rightmost element of the array as the pivot element.



Select a pivot element

2. Rearrange the Array

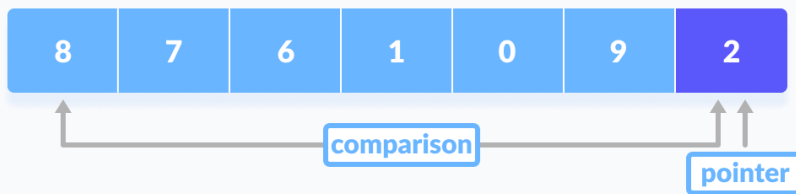
Now the elements of the array are rearranged so that elements that are smaller than the pivot are put on the left and the elements greater than the pivot are put on the right.



Put all the smaller elements on the left and greater on the right of pivot element

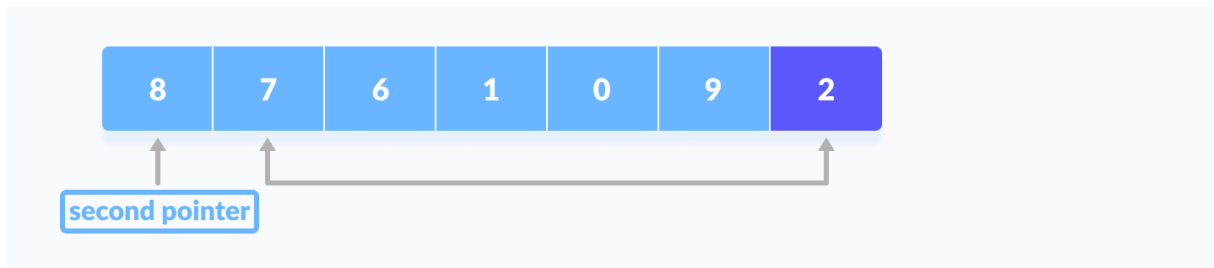
Here's how we rearrange the array:

1. A pointer is fixed at the pivot element. The pivot element is compared with the elements beginning from the first index.



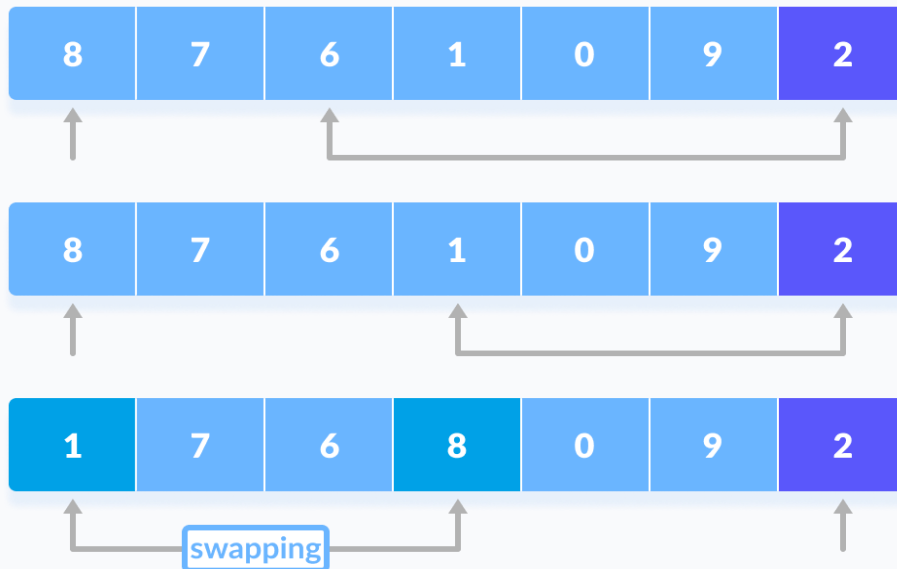
Comparison of pivot element with element beginning from the first index

2. If the element is greater than the pivot element, a second pointer is set for that element.



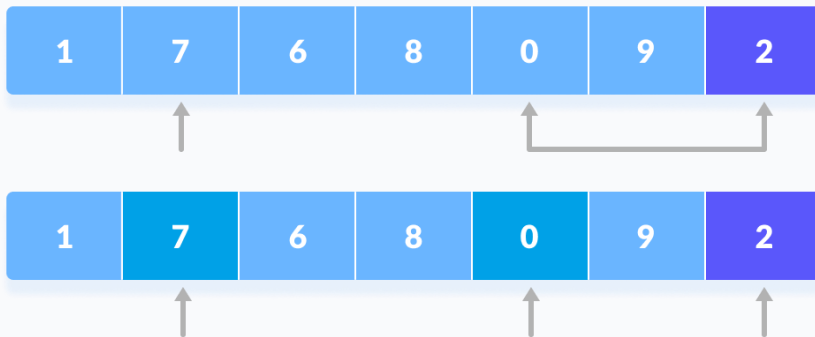
3. If the element is greater than the pivot element, a second pointer is set for that element.
4. Now, pivot is compared with other elements. If an element smaller than the pivot element is reached, the smaller element is swapped with the greater

element found earlier.



Pivot is compared with other elements.

5. Again, the process is repeated to set the next greater element as the second pointer. And, swap it with another smaller element.



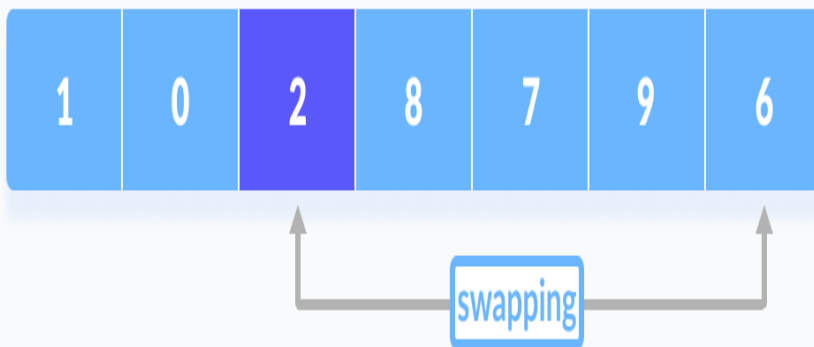
The process is repeated to set the next greater element as the second pointer.

6. The process goes on until the second last element is reached.



The process goes on until the second last element is reached.

7. Finally, the pivot element is swapped with the second pointer.

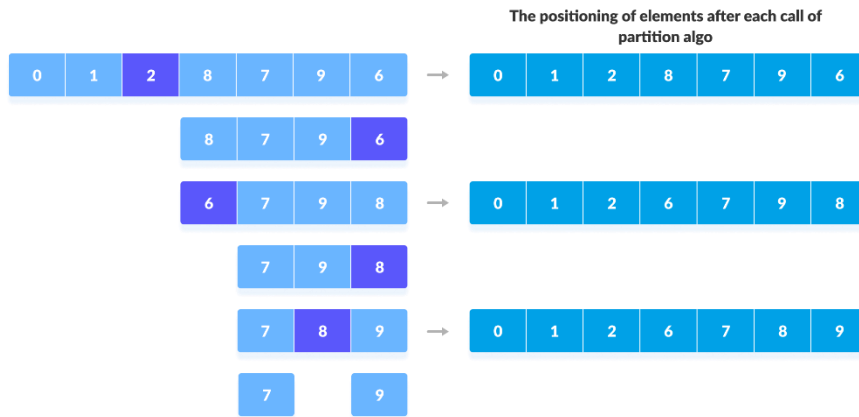


Finally, the pivot element is swapped with the second pointer.

3. Divide Subarrays

Pivot elements are again chosen for the left and the right sub-parts separately. And, **step 2** is repeated.

quicksort(arr, pi, high)



Select pivot element of in each half and put at correct place using recursion

The subarrays are divided until each subarray is formed of a single element. At this point, the array is already sorted.

Complexity Analysis of Quick Sort

Time Complexity:

- **Best Case:** ($\Omega(n \log n)$), Occurs when the pivot element divides the array into two equal halves.
- **Average Case** ($\theta(n \log n)$), On average, the pivot divides the array into two parts, but not necessarily equal.
- **Worst Case:** ($O(n^2)$), Occurs when the smallest or largest element is always chosen as the pivot (e.g., sorted arrays).

Auxiliary Space: $O(n)$, due to recursive call stack

Applications of Quick Sort

- Efficient for sorting large datasets with $O(n \log n)$ average-case time complexity.
- Used in partitioning problems like finding the kth smallest element or dividing arrays by pivot.
- Integral to randomized algorithms, offering better performance than deterministic approaches.
- Applied in cryptography for generating random permutations and unpredictable encryption keys.

- Partitioning step can be parallelized for improved performance in multi-core or distributed systems.
- Important in theoretical computer science for analyzing average-case complexity and developing new techniques.

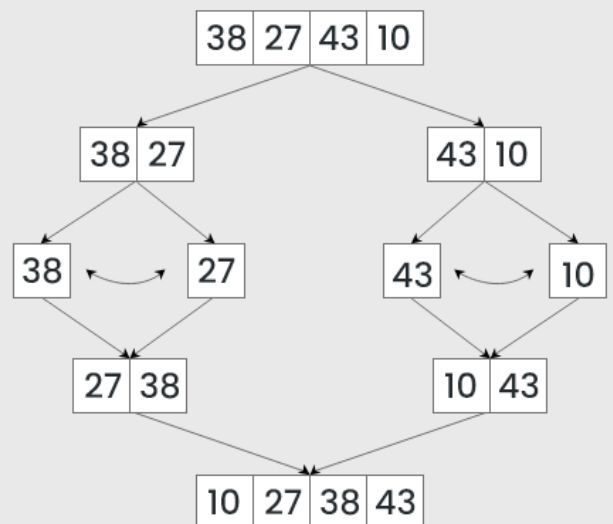
Merge Sort – Data Structure and Algorithms Tutorials

Merge sort is a sorting algorithm that follows the [divide-and-conquer](#) approach. It works by recursively dividing the input array into smaller subarrays and sorting those subarrays then merging them back together to obtain the sorted array.

In simple terms, we can say that the process of **merge sort** is to divide the array into two halves, sort each half, and then merge the sorted halves back together. This process is repeated until the entire array is sorted.

Merge Sort

Algorithm



Divide and Conquer Strategy

Using the **Divide and Conquer** technique, we divide a problem into subproblems. When the solution to each subproblem is ready, we 'combine' the results from the subproblems to solve the main problem.

Suppose we had to sort an array A . A subproblem would be to sort a sub-section of this array starting at index p and ending at index r , denoted as $A[p..r]$.

Divide

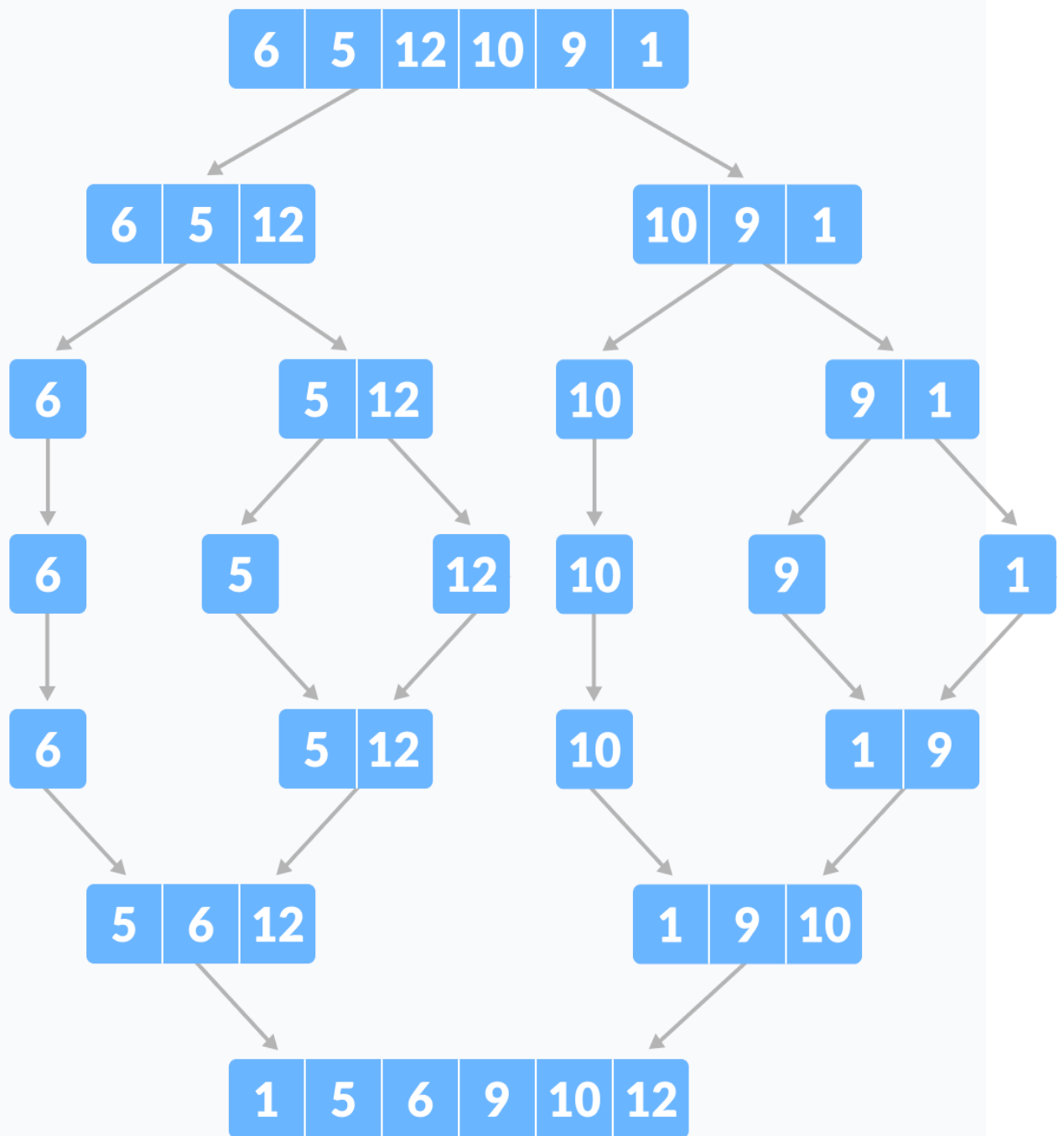
- If q is the half-way point between p and r , then we can split the subarray $A[p..r]$ into two arrays $A[p..q]$ and $A[q+1, r]$.

Conquer

- In the conquer step, we try to sort both the subarrays $A[p..q]$ and $A[q+1, r]$. If we haven't yet reached the base case, we again divide both these subarrays and try to sort them.

Combine

- When the conquer step reaches the base step and we get two sorted subarrays $A[p..q]$ and $A[q+1, r]$ for array $A[p..r]$, we combine the results by creating a sorted array $A[p..r]$ from two sorted subarrays $A[p..q]$ and $A[q+1, r]$.



Complexity Analysis of Merge Sort:

- Time Complexity:

- **Best Case:** $O(n \log n)$, When the array is already sorted or nearly sorted.
 - **Average Case:** $O(n \log n)$, When the array is randomly ordered.
 - **Worst Case:** $O(n \log n)$, When the array is sorted in reverse order.
- **Auxiliary Space:** $O(n)$, Additional space is required for the temporary array used during merging.

Applications of Merge Sort:

- Sorting large datasets
- External sorting (when the dataset is too large to fit in memory)
- It is a preferred algorithm for sorting Linked lists.
- It can be easily parallelized as we can independently sort subarrays and then merge.

Radix Sort – Data Structures and Algorithms

• **Radix Sort** is a linear sorting algorithm that sorts elements by processing them digit by digit. It is an efficient sorting algorithm for integers or strings with fixed-size keys.

Rather than comparing elements directly, Radix Sort distributes the elements into buckets based on each digit's value. By repeatedly sorting the elements by their significant digits, from the least significant to the most significant, Radix Sort achieves the final sorted order.

Radix Sort Algorithm

The key idea behind Radix Sort is to exploit the concept of place value. It assumes that sorting numbers digit by digit will eventually result in a fully sorted list. Radix Sort can be performed using different variations, such as Least Significant Digit (LSD) Radix Sort or Most Significant Digit (MSD) Radix Sort.

How does Radix Sort Algorithm work?

To perform radix sort on the array [170, 45, 75, 90, 802, 24, 2, 66], we follow these steps:

Consider this input

Array

170

45

75

90

802

24

2

66

Unsorted

Radix Sort



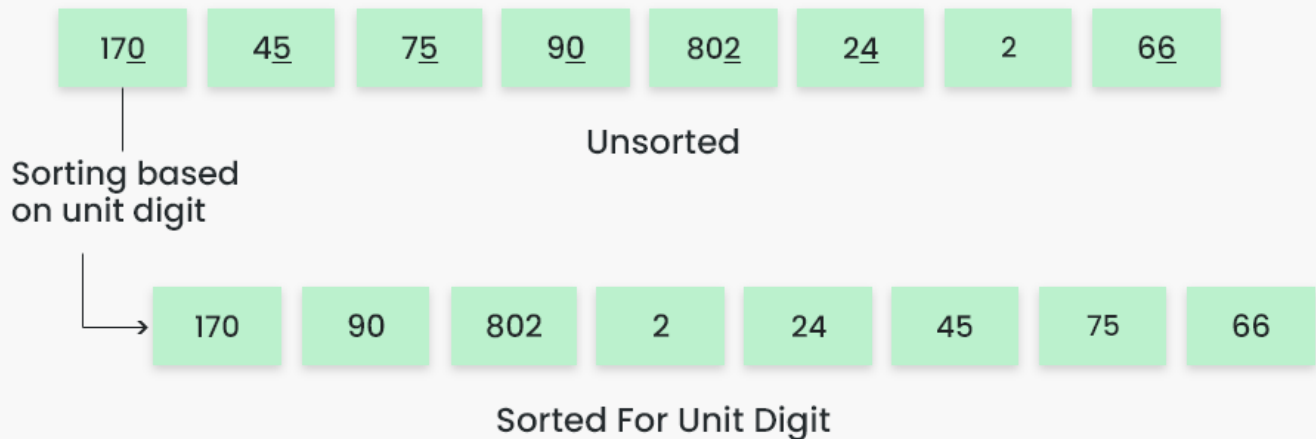
How does Radix Sort Algorithm work | Step 1

Step 1: Find the largest element in the array, which is 802. It has three digits, so we will iterate three times, once for each significant place.

Step 2: Sort the elements based on the unit place digits ($X=0$). We use a stable sorting technique, such as counting sort, to sort the digits at each significant place. It's important to understand that the default implementation of counting sort is unstable i.e. same keys can be in a different order than the input array. To solve this problem, We can iterate the input array in reverse order to build the output array. This strategy helps us to keep the same keys in the same order as they appear in the input array.

Sorting based on the unit place:

- Perform counting sort on the array based on the unit place digits.
- The sorted array based on the unit place is [170, 90, 802, 2, 24, 45, 75, 66].



Radix Sort

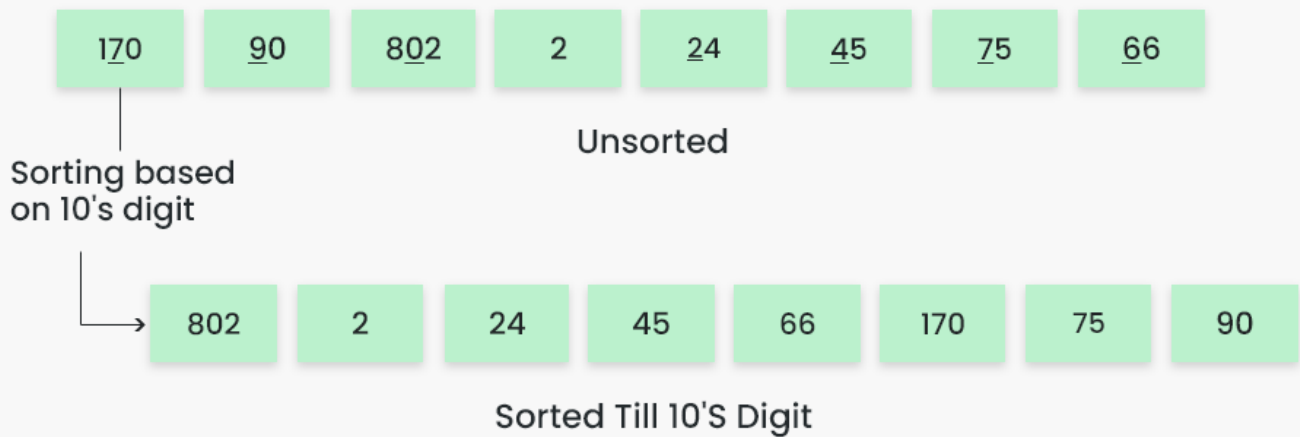


How does Radix Sort Algorithm work | Step 2

Step 3: Sort the elements based on the tens place digits.

Sorting based on the tens place:

- Perform counting sort on the array based on the tens place digits.
- The sorted array based on the tens place is [802, 2, 24, 45, 66, 170, 75, 90].



Radix Sort



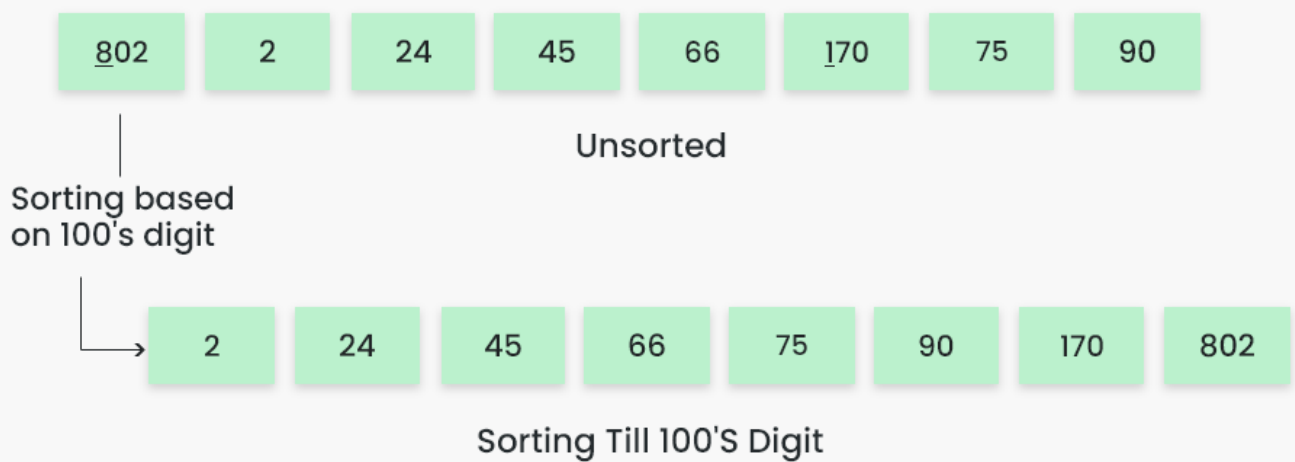
How does Radix Sort Algorithm work | Step 3

Step 4: Sort the elements based on the hundreds place digits.

Sorting based on the hundreds place:

- Perform counting sort on the array based on the hundreds place digits.

- The sorted array based on the hundreds place is [2, 24, 45, 66, 75, 90, 170, 802].



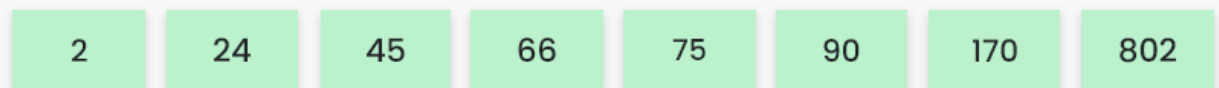
Radix Sort



How does Radix Sort Algorithm work | Step 4

Step 5: The array is now sorted in ascending order.
The final sorted array using radix sort is [2, 24, 45, 66, 75, 90, 170, 802].

Array after performing **Radix Sort** for all digits



Radix Sort



How does Radix Sort Algorithm work | Step 5

Shell Sort

- Shell sort is mainly a variation of Insertion Sort. In insertion sort, we move elements only one position ahead. When an element has to be moved far ahead, many movements are involved. The idea of ShellSort is to allow the exchange of far items. In Shell sort, we make the array h-sorted for a large value of h. We keep reducing the value of h until it becomes 1. An array is said to be h-sorted if all sublists of every h'th element are sorted.

Algorithm:

Step 1 – Start

Step 2 – Initialize the value of gap size, say h.

Step 3 – Divide the list into smaller sub-part. Each must have equal intervals to h.

Step 4 – Sort these sub-lists using insertion sort.

Step 5 – Repeat this step 2 until the list is sorted.

Step 6 – Print a sorted list.

Step 7 – Stop.

The interval between the elements is reduced based on the sequence used. Some of the optimal sequences that can be used in the shell sort algorithm are:

- **Shell's original sequence:** $N/2, N/4, \dots, 1$

Working of Shell Sort

1. Suppose, we need to sort the following array.

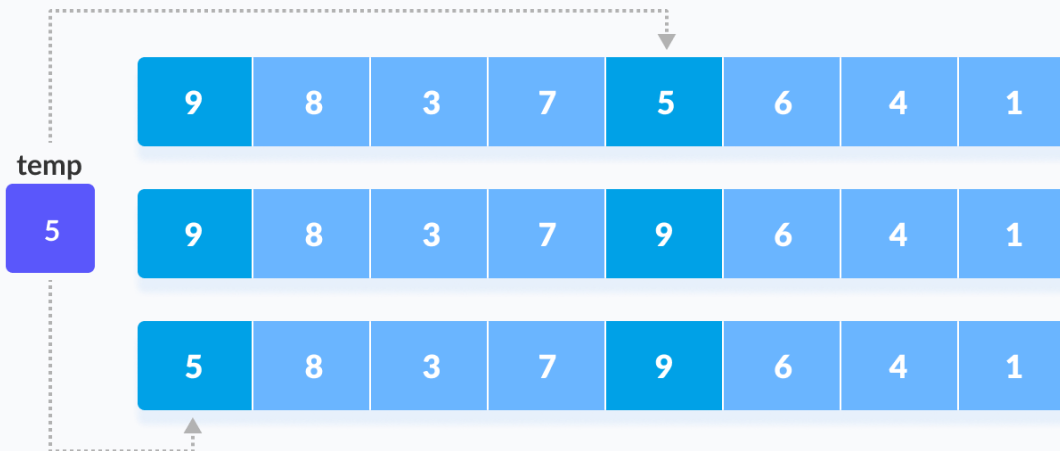
9	8	3	7	5	6	4	1
---	---	---	---	---	---	---	---

Initial array

2. We are using the shell's original sequence ($N/2, N/4, \dots, 1$) as intervals in our algorithm.

In the first loop, if the array size is $N = 8$ then, the elements lying at the interval of $N/2 = 4$ are compared and swapped if they are not in order.

- The 0th element is compared with the 4th element.
- If the 0th element is greater than the 4th one then, the 4th element is first stored in `temp` variable and the 0th element (ie. greater element) is stored in the 4th position and the element stored in `temp` is stored in the 0th position.



Rearrange the elements at $n/2$ interval

This process goes on for all the remaining elements.

5	8	3	7	9	6	4	1
---	---	---	---	---	---	---	---

5	6	3	7	9	8	4	1
---	---	---	---	---	---	---	---

5	6	3	7	9	8	4	1
---	---	---	---	---	---	---	---

5	6	3	1	9	8	4	7
---	---	---	---	---	---	---	---

Rearrange all the elements at $n/2$ interval

3. In the second loop, an interval of $N/4 = 8/4 = 2$ is taken and again the elements lying at these intervals are sorted.

5	6	3	1	9	8	4	7
---	---	---	---	---	---	---	---

3	6	5	1	9	8	4	7
---	---	---	---	---	---	---	---

Rearrange the elements at $n/4$ interval

You might get confused at this point.

3	1	5	6	9	8	4	7
---	---	---	---	---	---	---	---

3	1	5	6	9	8	4	7
---	---	---	---	---	---	---	---

All the elements in the array lying at the current interval are compared.

The elements at 4th and 2nd position are compared. The elements

at 2nd and 0th position are also compared. All the elements in the array lying at the current interval are compared.

4. The same process goes on for remaining elements.

3	1	5	6	9	8	4	7
3	1	5	6	9	8	4	7
3	1	4	6	5	8	9	7
3	1	4	6	5	8	9	7
3	1	4	6	5	7	9	8

Rearrange all the elements at $n/4$ interval

5. Finally, when the interval is $N/8 = 8/8 = 1$ then the array elements lying at the interval of 1 are sorted. The array is now completely sorted.

3	1	4	6	5	7	9	8
1	3	4	6	5	7	9	8
1	3	4	6	5	7	9	8
1	3	4	6	5	7	9	8
1	3	4	5	6	7	9	8
1	3	4	5	6	7	9	8
1	3	4	5	6	7	9	8
1	3	4	5	6	7	9	8
1	3	4	5	6	7	8	9

Rearrange the elements at $n/8$ interval

Shell Sort Complexity

Time Complexity

Best	$O(n \log n)$
Worst	$O(n^2)$
Average	$O(n \log n)$

Space Complexity

$O(1)$

Heap Sort

Heap sort is a sorting algorithm that sorts array elements using a heap data structure, which is similar to a binary tree data structure. The heap sort algorithm is essential to the data preparation process and can be used to implement priority queues, which sort items in a data structure based on assigned priority.

Understanding heap sort provides numerous benefits in domains such as data compression, path planning, robotics and automation.

Heap sort is a comparison-based sorting technique based on Binary Heap Data Structure. It can be seen as an optimization over selection sort where we first find the max (or min) element and swap it with the last (or first). We repeat the same process for the remaining elements. In Heap Sort, we use Binary Heap so that we can quickly find and move the max element in $O(\log n)$ instead of $O(n)$ and hence achieve the $O(n \log n)$ time complexity.

For now, we can understand heaps as binary trees with some special rules on top of them, which are:

1. **Complete binary trees:** A binary tree in which all levels are completely filled except the last one. And if the last level is filled partially, it should be filled from left to right.

2. **Max or min:** A heap should be either a max or a min heap. In a max heap, every parent, including the root, should be greater than or equal to its child nodes, so the largest value becomes the root. In a min heap, each parent node is less than or equal to its child nodes, so the smallest value of a node is at the root of the tree.

In theory, we could use both a max or a min heap to implement heap sort, but for demonstration purposes, we'll only implement it using max heap. You could always play around with the code and try implementing the algorithm with a min heap as well.

Working of Heap sort Algorithm

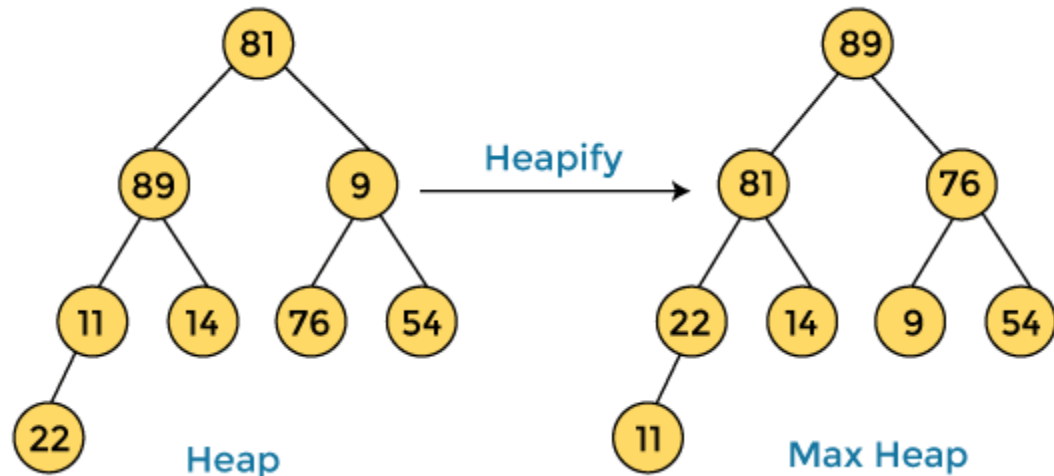
Now, let's see the working of the Heapsort Algorithm.

In heap sort, basically, there are two phases involved in the sorting of elements. By using the heap sort algorithm, they are as follows -

- The first step includes the creation of a heap by adjusting the elements of the array.
- After the creation of heap, now remove the root element of the heap repeatedly by shifting it to the end of the array, and then store the heap structure with the remaining elements.
- Now let's see the working of heap sort in detail by using an example. To understand it more clearly, let's take an unsorted array and try to sort it using heap sort. It will make the explanation clearer and easier.

81	89	9	11	14	76	54	22
----	----	---	----	----	----	----	----

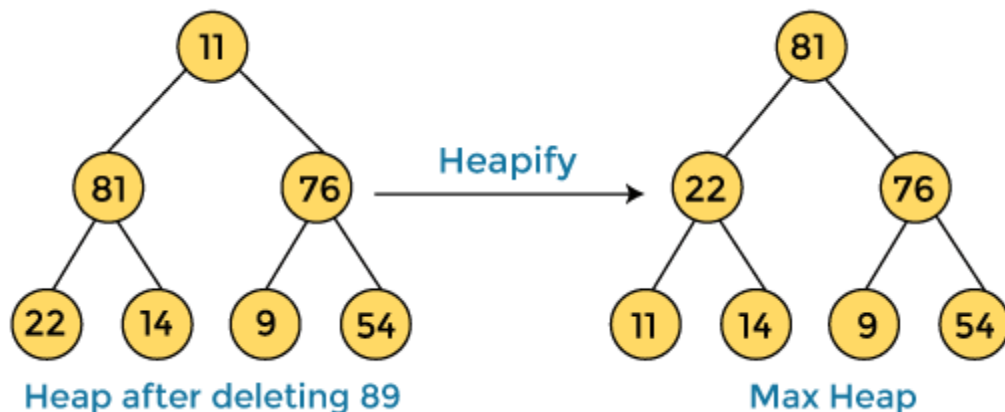
-
- First, we have to construct a heap from the given array and convert it into **maxa heap**.



- After converting the given heap into max heap, the array elements are -

89	81	76	22	14	9	54	11
----	----	----	----	----	---	----	----

- Next, we have to delete the root element (**89**) from the max heap. To delete this node, we have to swap it with the last node, i.e. (**11**). After deleting the root element, we again have to heapify it to convert it into max heap.

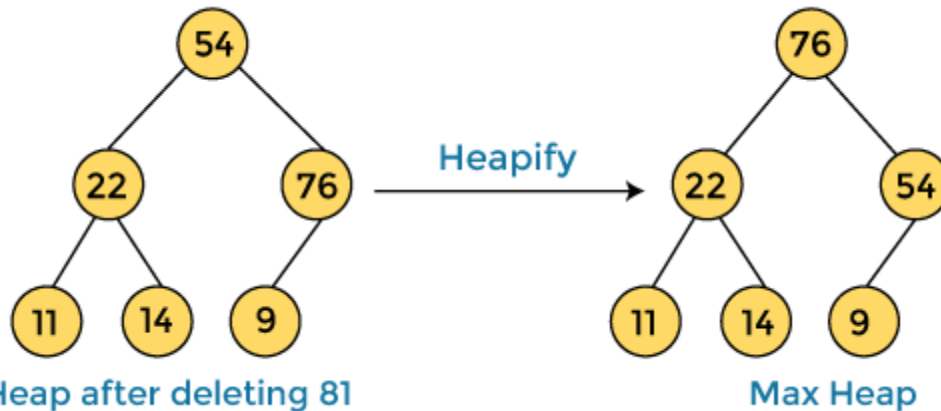


- After swapping the array element **89** with **11**, and converting the heap into max-heap, the elements of array are -

81	22	76	11	14	9	54	89
----	----	----	----	----	---	----	----

- In the next step, again, we have to delete the root element (**81**) from the max heap. To delete this node, we have to swap it with the last

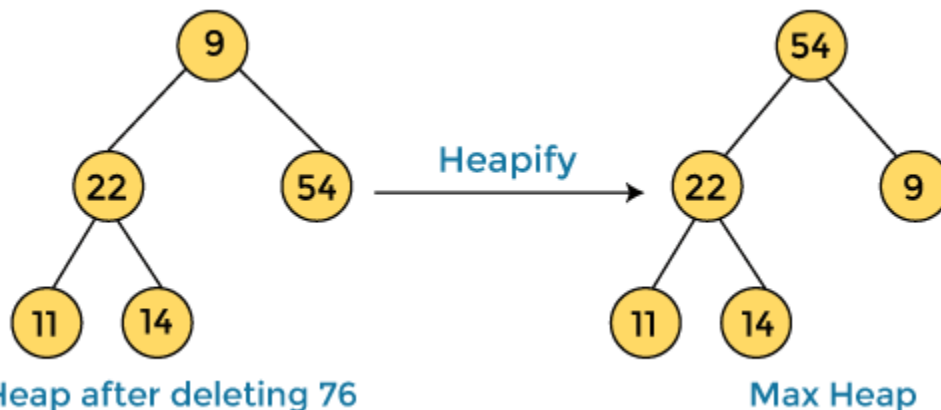
node, i.e. **(54)**. After deleting the root element, we again have to heapify it to convert it into max heap.



- After swapping the array element **81** with **54** and converting the heap into max-heap, the elements of array are -

76	22	54	11	14	9	81	89
----	----	----	----	----	---	----	----

- In the next step, we have to delete the root element **(76)** from the max heap again. To delete this node, we have to swap it with the last node, i.e. **(9)**. After deleting the root element, we again have to heapify it to convert it into max heap.

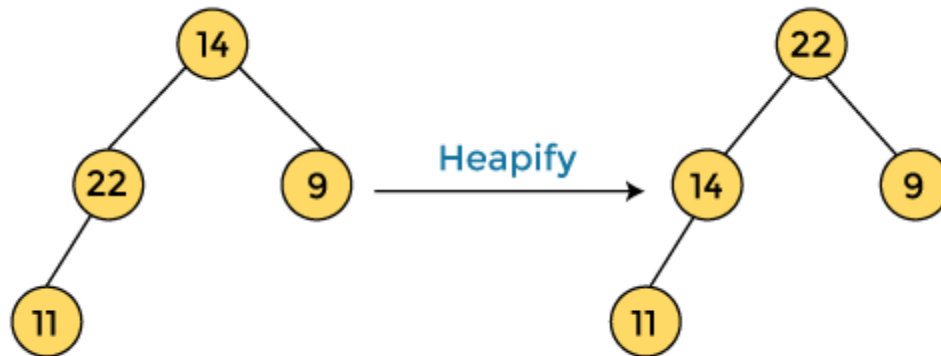


- After swapping the array element **76** with **9** and converting the heap into max-heap, the elements of array are -

54	22	9	11	14	76	81	89
----	----	---	----	----	----	----	----

- In the next step, again we have to delete the root element **(54)** from the max heap. To delete this node, we have to swap it with the last

node, i.e. **(14)**. After deleting the root element, we again have to heapify it to convert it into max heap.



Heap after deleting 54

Max Heap

-
- After swapping the array element **54** with **14** and converting the heap into max-heap, the elements of array are

22	14	9	11	54	76	81	89
----	----	---	----	----	----	----	----

-
- In the next step, again we have to delete the root element **(22)** from the max heap. To delete this node, we have to swap it with the last node, i.e. **(11)**. After deleting the root element, we again have to heapify it to convert it into max heap.



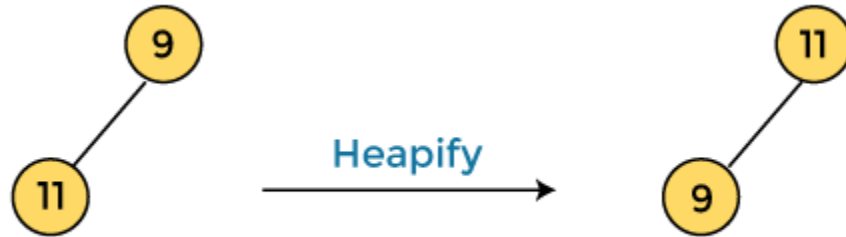
Heap after deleting 22

Max Heap

-
- After swapping the array element **22** with **11** and converting the heap into max-heap, the elements of array are -

14	11	9	22	54	76	81	89
----	----	---	----	----	----	----	----

-
- In the next step, again we have to delete the root element **(14)** from the max heap. To delete this node, we have to swap it with the last node, i.e. **(9)**. After deleting the root element, we again have to heapify it to convert it into max heap.



Heap after deleting 14

Max Heap

- After swapping the array element **14** with **9** and converting the heap into max-heap, the elements of array are -

11	9	14	22	54	76	81	89
----	---	----	----	----	----	----	----

- In the next step, again we have to delete the root element (**11**) from the max heap. To delete this node, we have to swap it with the last node, i.e. (**9**). After deleting the root element, we again have to heapify it to convert it into max heap.



Heap after deleting 11

Max Heap

- After swapping the array element **11** with **9**, the elements of array are -

9	11	14	22	54	76	81	89
---	----	----	----	----	----	----	----

- Now, heap has only one element left. After deleting it, heap will be empty.



- After completion of sorting, the array elements are -

9	11	14	22	54	76	81	89
---	----	----	----	----	----	----	----

- Now, the array is completely sorted.

Heap Sort Complexity

Time Complexity

Best	$O(n \log n)$
------	---------------

Worst	$O(n \log n)$
-------	---------------

Average	$O(n \log n)$
---------	---------------

Space Complexity	$O(1)$
------------------	--------